# Itara: Paying for a Team's Context Once

A Shared Open-Weight Gateway with Team Memory, a Permission-Scoped KV Cache and Confidence-Gated Escalation to a Closed Model

Prince Mutegetsi
pmuteget@caltech.edu

November 2026

## Abstract

Every engineer on a team sends a closed model much the same context (the same code, design notes and decisions) and the team pays for it on every question. Itara is a self-hosted gateway that the team shares instead. It compresses the team's material once into a memory that Postgres row-level security keeps to the team. It keys its KV cache by permission, so teammates reuse each other's prefill and other teams cannot. And it lets an open-weight model (Llama 3.1 8B) answer the questions it is sure of, escalating the rest to a closed model (Muse Spark). We evaluated Itara on the real models with all 300 SWE-bench Lite issues, posed as file localisation, and 300 LoCoMo questions. On code, the closed model given the team's memory is as accurate as with the full context (86.3% against 86.0%) while reading 8.9% of the context, about a quarter of the tokens and roughly half the estimated cost per question. The open model's confidence separates its right answers from its wrong ones (AUC 0.68 on code, 0.80 on conversations), and shared memory cuts ingest work per engineer by 35% at 16 engineers. Before the experiments we set a harder goal: match the closed model within 3 points while sending it at most 10% of its tokens. We have not reached it yet. At that budget the cascade scores 74.0% on code, though an oracle router passes, and stronger open models tried after the run raise the cascade to 80.3%. On conversations, retrieval rather than routing is what limits accuracy.

## 1 Introduction

A software team that adopts a frontier model through an API pays per token, and most of those tokens are context: the files around an issue, the design notes, the decisions made last month. Every engineer sends the same context with every question, and the team pays for it each time.

Two developments make a different arrangement possible. Open-weight models are now good enough to answer many routine questions themselves [18]. And serving systems can reuse the attention state of text they have already processed [5, 14, 17, 29, 31]. A team could therefore process its shared knowledge once, answer what it can locally, and send the closed model only what it must, in compact form.

Each of these ideas has been studied on its own. There is work on routers and cascades between cheap and expensive models [1, 3, 7, 22], on retrieval and memory for language models [15, 23], and on KV-cache reuse in serving [14, 29]. As far as we know they have not been combined and measured as one service that a team shares. That setting also raises an access-control question: a shared cache can leak one user's prompts to another [6, 24], so isolation has to be part of the design.

Itara puts these pieces into one gateway. A team ingests its code and documents once, and the open model compresses them into a memory every teammate retrieves from. The open model drafts each answer from that memory, reusing the team's cached prefill. When its own confidence is low, the question goes to the closed model with a few memory summaries in place of the raw context.

**The goal, set in advance.** Before running any experiment we wrote down a pass rule: *for a team sharing one codebase, the gateway matches the accuracy of sending the context straight to the closed model, within 3 points, while sending it at most 10% of the closed model's tokens.* We did not reach it, and Section 5.6 gives the verdict. After seeing the results we also ran analyses we had not planned: larger budgets, a lower reasoning effort, two further baselines and other open models. They are informative, but they were chosen after the fact, so they are marked wherever they appear and most are collected in Section 6.

**Contributions.**

- **The system.** Itara is an OpenAI-compatible gateway (Section 3) that combines a per-team compressed memory under Postgres row-level security, a KV cache whose keys include the reader's permission scope, a confidence-gated cascade to a closed model, and exact token accounting with a hard spending cap.
- **A pre-registered evaluation** on the real models: all 300 SWE-bench Lite issues [10] as file localisation and 300 LoCoMo questions [19], against six baselines and an oracle router, with gates scored out of fold and paired bootstrap

intervals (Section 4).

- **What works.** On code, compressed team memory lets the closed model match its full-context accuracy with under a tenth of the context. The open model's confidence is a usable escalation signal. Sharing memory reduces each engineer's work as the team grows, and keying the cache by permission removes the cross-team timing channel by construction.
- **Where the goal stands.** We show what keeps the 10% goal out of reach today. With a reasoning model, the closed model's own output, not the context, sets the cost of an escalation. The 8B open model's accuracy limits any gate that judges its drafts. On conversations, retrieval caps every routing policy. Stronger open models, tried after the run, narrow the gap on code (Section 6.3).

## 2 Related work

**Routing and cascades.** FrugalGPT [1] chains models of increasing cost and stops when a scorer accepts an answer. Hybrid LLM [3] and RouteLLM [22] learn a router that predicts, before generation, whether the small model will do. Gupta et al. [7] show that sequence-level confidence is biased by length and that token-level quantiles defer better. Itara is a cascade in this sense, but it saves in a different place. Its escalations carry compact team memory instead of raw context, and its gate is scored against budgets measured on the closed model's actual token usage, reasoning included.

**Knowing when a model is wrong.** A model's own probabilities carry information about whether it is right [11], and agreement between sampled answers is a strong black-box signal [13, 20, 26]. Our cascade gate combines token log-probabilities, agreement among four resamples and a yes/no self-check, all computed on the open model's cached prefix.

**Memory and context.** Retrieval-augmented generation [15] and agent memory systems such as MemGPT [23] keep knowledge outside the prompt. Long prompts cost more and can also hurt accuracy [8, 16], and prompt compression cuts tokens directly [9]. Itara's memory belongs to a team rather than a user. The open model compresses it once, and it is searched by vector and keyword search fused by reciprocal rank [2].

**KV-cache reuse and its risks.** vLLM's paged attention and prefix caching [14], SGLang's radix tree [31], Prompt Cache [5], CacheBlend [29] and LMCache [17] all reuse attention state across requests. Shared caches leak. Response timing reveals whether another user's prompt was cached [24], and audits have found cross-user cache sharing at several commercial APIs [6]. Itara keys the cache by permission scope, so a hit is only possible between readers with the same access.

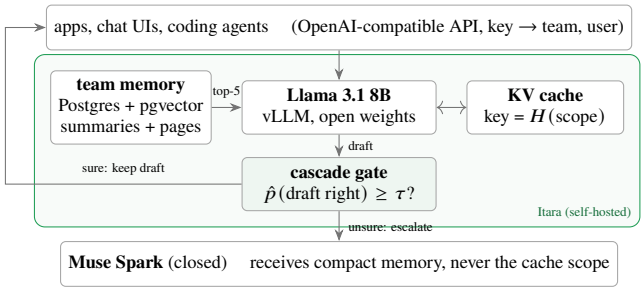



Figure 1: Itara. The open model drafts every answer from the team's memory, reusing the team's cached prefill. The gate keeps confident drafts and escalates the rest to the closed model with compact memory. Row-level security and scope-keyed caching keep teams apart.

## 3 System

Itara is a single server (Figure 1) that speaks the OpenAI chat-completions API, so a client adopts it by changing a base URL. It runs the open model on vLLM [14], stores memory in Postgres with pgvector [12], and reaches the closed model through the Meta Model API [21].

**Identity and isolation.** Each API key maps to one team and one user, and only its hash is stored. Identity never comes from client headers. Every memory row carries a team and, optionally, an owner. Postgres row-level security [25], forced even for the table owner, limits every query to rows of the caller's team that are either shared or owned by the caller. Personal memory is private by default; code, documents and decisions are shared with the team.

**Shared memory.** A document is split on blank lines into pages of at most 2,000 characters. The open model compresses each page into a JSON summary, one fact per sentence (who did what, and when), plus keywords. If it fails twice, the page's opening text is stored instead. Summaries are embedded with BGE-small [28], and the raw pages are kept behind them. A page the team already holds is never compressed again, a changed file replaces its old version, and a source can be removed. A question retrieves its top $k$=5 summaries by reciprocal rank fusion [2] of a cosine-similarity ranking and a full-text ranking:

$$\text{score}(m) = \sum_{r \in \{\text{vec},\text{text}\}} \frac{1}{60 + \text{rank}_r(m)}, \qquad (1)$$

with ties broken by id so that retrieval is deterministic. Code sources are labelled with their commit and conversation sources with their session date, which the compression prompt uses to resolve relative times.

**Permission-scoped KV cache.** A request to the open model carries a cache salt $s = H(\text{team})$ if all the memory it retrieved is shared, or $H(\text{team} \,\|\, \text{user})$ if any of it is private.

vLLM mixes the salt into the hash of the first KV block, so prefix hits only happen between requests with the same scope. A teammate asking about the same code reuses the prefill. A member of another team never hits the cache, even with an identical prompt, and so cannot learn anything from its timing [24]. The closed model receives memory but never the salt.

**Routes and modes.** Each question takes one of three routes. On *local_memory* the open model answers with the top-5 summaries; on *escalate_compact* the closed model answers with the same summaries; on *escalate_raw* it answers with the raw pages behind them. The gateway runs in one of three modes. Accurate mode sends every question to the closed model with memory, fast mode lets the cascade gate below decide, and local mode never calls the closed model. Requests with tool calls, which is what coding agents send, skip the gate. They receive the team's memory as a system message and go on with their tools unchanged.

**Gates.** The *rule gate* is a baseline. It escalates questions longer than 200 tokens or containing a fenced code block. The *per-route gate* fits, for each route, a logistic model of $P(\text{right} \mid x)$ on 18 features of the question, the retrieved memory and the open model's draft, and picks

$$\hat{r}(x) = \arg\max_r \; \hat{P}_r(\text{right} \mid x) - \lambda \, \widehat{\text{cost}}_r(x), \qquad (2)$$

where the cost is estimated in dollars before the call. Sweeping $\lambda$ traces a cost–accuracy curve. Its training labels are each route's actual correctness on the item. The *cascade gate*, which Itara ships, is simpler. The open model always drafts first, and a single logistic score $\hat{p}$ that the draft is right is computed from six confidence features:

- the mean and the minimum token log-probability of the draft;
- the share of its tokens with probability below one half;
- how well four resamples at temperature 1 agree with the draft, by cosine similarity and by exact first-line match;
- the probability of "yes" when the model is asked whether its answer is correct and supported by the memory.

The resamples and the self-check reuse the draft's cached prefix, so they add little prefill. If $\hat{p} < \tau$ the question escalates, with summaries when most of the retrieved memory is code and with raw pages otherwise.

**Accounting and the cap.** Every model call is recorded with the token counts the model reports, tagged with the request it belongs to. Before a paid call, its worst-case cost is reserved under a lock against a hard cap, so concurrent requests cannot overrun it, and the call's output is limited to what was reserved. An admin view shows the routes taken, cached tokens reused and spend per person and per team. The release includes a live dashboard and a command-line tool.

# 4 Experimental setup

**Models and infrastructure.** The open model is Llama 3.1 8B Instruct [18], served by vLLM with prefix caching on one H100 on Modal. The closed model is Muse Spark 1.2 through the Meta Model API [21]. It is a reasoning model, so its output tokens include its reasoning, and we use its default effort unless stated. Answers are generated at temperature 0.

**Benchmarks.** For code we use all 300 SWE-bench Lite issues [10], posed as file localisation, a step recent agent pipelines rely on [27]. The model names at most five files to change, and an answer counts as right only if every file in the gold patch is among them, so no judge is needed. Each repository is a team, and its memory is the files in the BM25-retrieved context distributed with the benchmark (about 13K tokens per issue). For conversations we use 300 questions from LoCoMo [19] (categories 1–4, sampled with a fixed seed). Each conversation is a team and each session a memory source. The closed model judges these answers at minimal effort [30]. We checked 50 randomly sampled verdicts by hand, across conditions, and agreed with the judge on 48.

**Conditions.** *Muse, full context* sends the closed model all of the retrieved code (11,253 prompt tokens on average) or the whole conversation (18,328). It is the accuracy reference and the denominator for every token share. The other conditions are *always escalate* (with summaries or with raw pages), the *rule gate*, *Llama + memory* (never escalate) and Itara's *cascade gate*. The *oracle router* knows every route's correctness and picks the cheapest right one, so no gate choosing among these routes can beat it.

**Metrics and protocol.** Accuracy is the share of items answered right. Cost is the closed model's tokens, prompt plus completion, as a share of the reference's on the same items; we also report the prompt-only share. Gates are trained and scored out of fold, with five folds grouped by team so that a repository or conversation never appears in both training and test. Intervals on accuracy gaps are paired bootstrap 95% intervals over items, with 4,000 resamples [4]. The pass rule is applied to point estimates at each benchmark's 10% operating point.

**Team replay.** To measure sharing, we replay SWE-bench tasks on one repository with $n \in \{1, 2, 4, 8, 16\}$ simulated engineers doing two tasks each, either as one team (shared memory and cache) or each working alone. Tasks are ordered either by overlap with files the team has already touched (*related*) or at random. Pages ingested per engineer measures the gateway's work independently of timing.

Table 1: Main results. Gap: accuracy minus Muse with the full context, in points, with a paired bootstrap 95% interval. Token share: Muse prompt + completion tokens relative to the reference on the same items; prompt share counts prompt tokens only. Kept: questions answered by the open model. On LoCoMo the rule gate never escalates (the questions are short and contain no code blocks), so it matches Llama + memory.

| Condition | Accuracy (%) | Gap (pts) [95% CI] | Token share (%) | Prompt share (%) | Kept (%) |
|---|---|---|---|---|---|
| *SWE-bench Lite (code)*, 300 items | | | | | |
| Muse, full context | 86.0 | – | 100.0 | 100.0 | 0 |
| Itara, cascade gate at 10% | 74.0 | −12.0 [−17.0, −7.3] | 10.0 | 3.6 | 61 |
| Muse + raw pages (always) | 87.0 | +1.0 [−1.3, +3.3] | 34.1 | 26.0 | 0 |
| Muse + summaries (always) | 86.3 | +0.3 [−2.3, +3.0] | 23.0 | 8.9 | 0 |
| Rule gate | 82.7 | −3.3 [−6.7, +0.0] | 21.0 | 8.0 | 17 |
| Llama + memory (never escalate) | 55.7 | −30.3 [−36.3, −24.7] | 0.0 | 0.0 | 100 |
| *Oracle router (upper bound)* | 90.3 | +4.3 | 8.8 | 3.6 | 65 |
| *LoCoMo (conversations)*, 300 items | | | | | |
| Muse, full context | 87.0 | – | 100.0 | 100.0 | 0 |
| Itara, cascade gate at 10% | 54.7 | −32.3 [−38.0, −26.3] | 9.4 | 6.6 | 41 |
| Muse + raw pages (always) | 62.7 | −24.3 [−29.7, −19.0] | 15.7 | 11.3 | 0 |
| Muse + summaries (always) | 46.0 | −41.0 [−47.0, −35.0] | 7.7 | 2.6 | 0 |
| Rule gate | 38.3 | −48.7 [−54.7, −42.7] | 0.0 | 0.0 | 100 |
| Llama + memory (never escalate) | 38.3 | −48.7 [−54.7, −43.0] | 0.0 | 0.0 | 100 |
| *Oracle router (upper bound)* | 66.7 | −20.3 | 3.6 | 2.3 | 72 |

# 5 Results

## 5.1 On code, team memory keeps the closed model's accuracy

Table 1 and Figure 2 cover every condition. Sending every question to the closed model with the top-5 memory summaries scores 86.3%, against 86.0% with the full context (gap +0.3, interval −2.3 to +3.0). We see no measurable loss. The closed model reads 8.9% of the prompt tokens it would otherwise receive, 1,004 instead of 11,253 per question. Giving it the raw pages behind the summaries instead reaches 87.0% at 26.0% of the prompt.

The total token share of the summary route is 23.0%, though, not 9%. Muse writes about 2,822 tokens of reasoning and answer per escalation, however short its prompt. With a reasoning model, compression removes the cost of the context but leaves the model's own output as a floor.

Muse has very likely seen these public repositories in training, which makes code localisation a lenient test of memory. The reference names the right files for 80% of the 201 issues whose gold file was not in its retrieved context at all. To measure what the memory itself adds, we ran two more baselines on the same issues after the main results. Given only the issue and no context, Muse localises 81.3% (244/300). Team memory adds +5.0 points to that (259/300, interval +2.0 to +8.0) and is also cheaper, $16.11 against $20.64 per thousand questions, because Muse reasons for longer when it has no context. On a private codebase the model would know nothing in advance, and memory would be the only source. LoCoMo, where the answers exist only in the team's conversations, is the strict test (Section 5.5).

The second baseline asks whether summaries beat plain retrieval. Searching the raw pages directly with the same hybrid search, with no summaries and nothing to build, scores 85.3% (256/300), against 86.3% for summaries (gap +1.0, interval −1.7 to +3.3). Including the cost of building the memory for these 300 questions, the two cost about the same ($15.79 and $16.11 per thousand). Per question, though, summaries send Muse a third of the prompt (1,004 tokens against 3,027) and cut its bill by 16%, so the build pays for itself after about 338 questions on the same memory. Plain retrieval is a strong baseline at this scale. Summaries match its accuracy, cost less as a team keeps asking, and are what the open model reads in fast and local modes.

Table 2: How well each signal separates the open model's right drafts from wrong ones (AUC, out of fold; 0.5 is chance).

| Signal | Code | Conv. |
|---|---|---|
| 4 resamples give the draft's answer | 0.68 | 0.71 |
| Similarity of 4 resamples to the draft | 0.60 | 0.70 |
| Mean token log-probability | 0.60 | 0.74 |
| Minimum token log-probability | 0.58 | 0.70 |
| Self-check: P(yes) that the draft is right | 0.61 | 0.66 |
| Best question–memory similarity | 0.63 | 0.55 |
| Cascade score (6 confidence features) | 0.65 | 0.74 |
| Per-route gate, P(draft right) (18 features) | 0.68 | 0.80 |

## 5.2 The gate has a real signal but a hard budget

The open model's confidence is informative (Table 2). The strongest single feature is whether four resamples repeat the

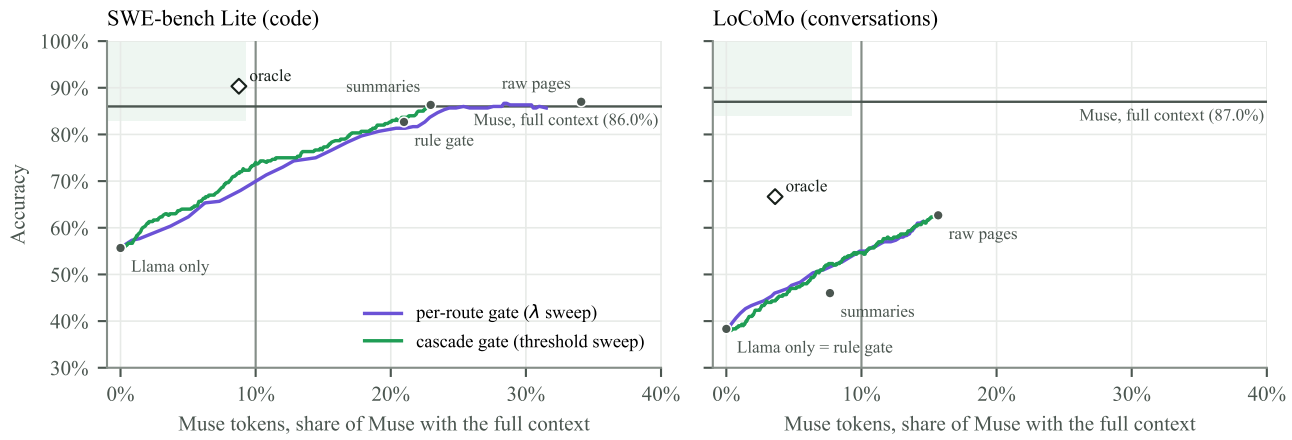



Figure 2: Accuracy against the closed model's tokens. Lines sweep each gate's threshold or $\lambda$ (out of fold). Dots are fixed routes; the diamond is the oracle router. The shaded box is the pre-registered pass region: within 3 points of the full-context reference at $\leq$10% of its tokens.

Table 3: Gates at each benchmark's 10% budget (out of fold). Random: escalating the same share of tokens at random. The per-route gate uses 18 features and the cascade gate six confidence features. The last two rows are upper bounds.

| Gate | Acc. (%) | Tokens (%) | Random (%) |
|---|---|---|---|
| *Code* | | | |
| Per-route gate | 68.0 | 8.9 | 67.5 |
| Cascade gate | 74.0 | 10.0 | 69.0 |
| Perfect draft check | 88.3 | 12.1 | – |
| Oracle router | 90.3 | 8.8 | – |
| *Conversations* | | | |
| Per-route gate | 55.0 | 9.9 | 53.7 |
| Cascade gate | 54.7 | 9.4 | 53.0 |
| Perfect draft check | 65.7 | 10.0 | – |
| Oracle router | 66.7 | 3.6 | – |

draft's answer, and mean token log-probability and the self-check also carry signal. All of them come from the open model alone, with no calls to the closed model. At the 10% budget (Table 3), the cascade gate scores 74.0% on code against 69.0% for random escalation with the same tokens, and keeps 61% of questions on the open model.

The per-route gate does worse at the budget (68.0%), despite a higher AUC for its draft score. It fits 18 features for each of three routes on about 480 training items per fold, and decides on the difference between two noisy probabilities. On conversations only 38% of drafts are right, so a 10% budget already escalates most questions and ranking them matters little.

Two upper bounds show why 10% is out of reach for any gate that judges only the draft. A perfect draft check, which escalates exactly the wrong drafts, would score 88.3% on code but needs 12.1% of the tokens. The questions a good gate escalates are also the expensive ones: the closed model writes 4,543 tokens on questions the open model got wrong, against 3,256 on those it got right. Only the oracle passes on code (90.3% at 8.8%), because it also knows when the closed model will fail.

## 5.3 What a question costs

Token shares leave out two costs, the GPU that runs the open model and the time a user waits. Table 4 prices each option on the same 300 code questions from the saved answers, so this analysis calls no model. Muse is charged its exact tokens at list price. The GPU is charged at Modal's listed H100 rate for each request's open-model time, as if the request had the GPU to itself. That overstates it, since the deployment serves up to 32 requests at once. Every option that reads the memory also pays for building it: 13 minutes of H100 time for these issues, or $2.86 per thousand questions, which a team asking more questions of the same code pays only once.

Team memory cuts the cost per question by 57% at the same accuracy, and answers slightly sooner. The cascade cuts a further 26% (68% in all). Its median answer arrives in 2.3 s instead of 9.8 s because most questions never leave the GPU, though at this budget it scores 74.0% against 86.3%. Per correct answer it costs a little less than memory alone ($16.04 against $18.66 per thousand). At a 20% budget it costs more than memory alone and is less accurate. Escalating the same number of questions at random is cheaper still but scores only 67.5%. The offline replay charges the cascade's confidence calls the draft's time again, since they were not timed.

So after the results we also ran fast mode live in the gateway, at the 10% threshold, six questions at a time, timing each question end to end: draft, confidence calls and any escalation. It scores 72.7% with a median wait of 1.9 s (90th percentile 32.4 s), escalating 37% of questions, at $11.34 per

Table 4: The cost of 1,000 code questions, closed-model tokens and GPU time together, on the same 300 SWE-bench Lite issues; latency is per question. Prices: Muse's list prices, and an H100 at $3.95 an hour charged as if each request had it alone, memory build included. Muse + raw pages: the pages behind the retrieved memories, without summaries. Random: as many escalations as the cascade at 10%, mean of 1,000 draws. RouteLLM: an off-the-shelf router [22] deciding from the question alone. †Budget chosen after the run. ‡Added after the results (Sections 5.1 and 5.3). The live cascade's latency is measured end to end with six questions in flight; the two other ‡ rows were answered eight at a time, so their latencies are not comparable with the rest.

| Option | Acc. (%) | $/1k | $/1k right | Median (s) |
|---|---|---|---|---|
| Muse, full context | 86.0 | 37.09 | 43.12 | 12.2 |
| Muse, question only‡ | 81.3 | 20.64 | 25.38 | 17.0 |
| Raw pages, direct‡ | 85.3 | 15.79 | 18.50 | 16.4 |
| Muse + raw pages | 87.0 | 18.23 | 20.96 | 9.9 |
| Muse + team memory | 86.3 | 16.11 | 18.66 | 9.8 |
| Rule gate | 82.7 | 15.25 | 18.45 | 8.6 |
| Random escalation | 67.5 | 9.56 | 14.18 | 1.1 |
| Cascade, 10% | 74.0 | 11.87 | 16.04 | 2.3 |
| Cascade, 20%† | 82.7 | 17.57 | 21.25 | 11.0 |
| Cascade 10%, live‡ | 72.7 | 11.34 | 15.61 | 1.9 |
| RouteLLM, 10% | 68.3 | 9.85 | 14.41 | 1.3 |
| RouteLLM, 20%† | 82.3 | 14.59 | 17.72 | 8.7 |

thousand. The live gate re-runs the confidence probes, so it does not decide exactly as the replay does, but this is the number a deployment sees.

The simpler alternatives are close on code. Muse given raw pages, without summaries, costs $18.23 per thousand for 87.0%. Once the closed model reasons for thousands of tokens per answer, shortening its prompt further saves little. The rule gate costs about the same as memory alone and is less accurate (82.7% against 86.3%).

**Against an off-the-shelf router.** RouteLLM's BERT router [22], trained on Chatbot Arena preferences and used as shipped, decides from the question alone whether the strong model is needed. We send the questions it picks straight to the closed model with team memory, and set its threshold to the most accurate one within each budget on these items, which favours it. Its score barely predicts when the open model's draft is wrong (AUC 0.53 on code and 0.55 on conversations, against the cascade's 0.65 and 0.74). At the 10% budget on code that difference is worth points: the cascade scores 74.0% and the router 68.3%. The router is cheaper and faster, because it never drafts the questions it escalates. At 20% the two are equally accurate (82.7% and 82.3%) and the router costs less, and on conversations at 10% they are level (54.7% and 54.3%). The cascade's advantage is its signal, and that matters most when the budget is tight.

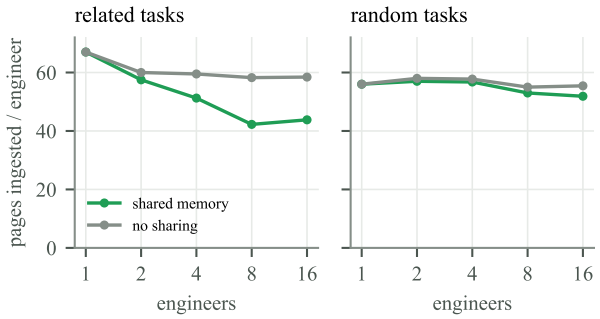



Figure 3: Pages ingested per engineer as the team grows (SWE-bench, one repository, two tasks per engineer).

**Sensitivity.** Two of the reference's answers reasoned up to the closed model's 262,144-token output limit and returned nothing. Dropping them from every option, team memory costs $15.98 per thousand against $29.75, 46% less rather than 57%. Muse's API also bills cached prompt tokens at $0.15 per million instead of $1.25. On code it served 0% of the full-context prompts from its cache, since each issue's context differs, so nothing changes there. On conversations, where many questions share one transcript, 93% was cached. Repriced at the cached rate, full context costs $8.58 per thousand against $5.47 for team memory, and it is far more accurate on that benchmark.

Taken together, team memory justifies itself on cost alone. With an 8B open model, the cascade earns its extra complexity through latency and a lower bill, at some cost in accuracy, and it should earn more as the open model gets more questions right (Section 6.3).

## 5.4 Sharing across a team

Figure 3 shows each engineer's share of the work as the team grows. With shared memory, pages ingested per engineer fall by 35% at 16 engineers on related tasks, against 13% when each works alone. On random tasks the drop is 7% against 1%. The scoped KV cache behaved as designed on the H100 deployment. With one 6,150-token team memory, a teammate's request reused 6,144 cached prompt tokens and a request from another team reused none. On an H100 the wall-clock saving for 6K tokens is small (0.43 s against 0.59 s, mostly network round trips), so reused tokens are the better measure. The ingest saving is also small in dollars: at the measured build rate, ingest costs $0.0062 per engineer for two tasks with sharing and $0.0083 alone, against about a cent of closed-model spend per question. At this scale sharing saves work and waiting more than money.

## 5.5 On conversations, retrieval is the bottleneck

On LoCoMo the best route, raw pages for every question, reaches 62.7% at 15.7% of the tokens, against 87.0% for

Table 5: The most accurate option within each budget (any fixed route, either gate's sweep). †Budget chosen after seeing the results. ✓: within 3 points of the reference.

| Budget | Acc. (gap) | Option | Tokens (%) |
|---|---|---|---|
| *Code* | | | |
| 10% | 74.0 (−12.0) | cascade gate | 10.0 |
| 20%† | 82.7 (−3.3) | cascade gate | 19.9 |
| 25%† | 86.3 (+0.3) ✓ | all to Muse, summaries | 23.0 |
| *Conversations* | | | |
| 10% | 55.0 (−32.0) | per-route gate | 9.9 |
| 20%† | 62.7 (−24.3) | cascade gate | 15.6 |
| 25%† | 62.7 (−24.3) | cascade gate | 15.6 |

the reference. Splitting the questions by whether retrieval found a memory from the session that holds the evidence (it did for 63%) shows where the loss comes from. When retrieval found it, the closed model with raw pages scores 85.3% against the reference's 88.9%, while summaries alone score 60.5%. Compression loses detail here, and the raw pages restore it. When retrieval missed, every gateway route scores 20.9–23.6%, while the reference, which reads everything, scores 83.6%. The oracle router reaches only 66.7%, so with this retrieval no routing policy can pass.

### 5.6 The 10% goal

At 10% of the closed model's tokens, the cascade gate reaches 74.0% on code against 86.0% (gap −12.0 points, interval −17.0 to −7.3), and 54.7% on conversations against 87.0%. Neither meets the pre-registered rule yet. On code the gate's upper bounds explain the gap, and on conversations retrieval does. Section 7 discusses what would close it.

## 6 Analyses chosen after the run

The analyses in this section were chosen after we had seen the results. We report them with intervals and do not count any of them as meeting the goal.

### 6.1 Larger budgets

A team may prefer accuracy to the deepest saving. Table 5 adds 20% and 25% budgets as points on the curve. On code, sending every question with summaries matches the reference at 23.0% of its tokens. The two runaway reference answers make up 11% of its tokens, so we describe this as about a quarter. At that budget most of the saving comes from compact memory rather than from answering locally. Conversations stop at 62.7% whatever the budget.

### 6.2 Lower reasoning effort

The closed model's output dominates the cost of an escalation, so we reran every closed-model condition at minimal reasoning effort on the same memory and drafts, with a full-context reference at the same effort (Table 6). Output per escalation falls from 2,822 to 336 tokens. On code the per-route gate then scores 80.7% against 83.0% (gap −2.3, interval −5.3 to +0.7) at 9.8% of the tokens, keeping 13% of questions on the open model. Choosing $\lambda$ on the other folds gives a similar result, so in-sample selection does not explain it.

The point estimate falls inside the pass rule, but this is not a pass. The interval admits misses of over 5 points (the gap is at least −3 in 67% of resamples), the setting was chosen after the run, and the reference itself is weaker than at default effort. Against the default-effort reference the same gate misses by 5 points. Sending every question with summaries at minimal effort scores 83.3%, a gap of −2.7 points to the *default-effort* reference at 8.0% of its tokens. Conversations do not move (57.3%).

### 6.3 Other open models

To test whether the open model limits the gate, we swapped the drafting model and kept everything else fixed: the same questions, the same team memory (built by the 8B), the closed model's answers from the full run for every escalation, and the same out-of-fold gate training (Table 7).

**Llama 3.3 70B.** On code, the 70B's drafts are right on 67.0% of issues against 55.7% for the 8B (gain +11.3 points, interval +6.7 to +16.0). The cascade at 10% scores 77.7% against 74.0% (gain +3.7, interval −0.7 to +8.0; better in 94% of resamples), still −8.3 points from the reference. At a 20% budget it scores 85.7% at 17.1% of the tokens, within 0.3 points of the reference, keeping 26% of questions on the open model. A perfect draft check now passes: escalating exactly the 33% of drafts that are wrong would score 90.0% at 9.9% of the tokens.

What remains is the gate. Its signal separates the 70B's right and wrong drafts less well than the 8B's (AUC 0.58 against 0.65). Some of this is measurement noise. Redrafting at temperature 0 under batching reproduced the 70B's answer for only 60% of items (89% for the 8B), so its confidence features describe a slightly different draft from the one that was scored. The 70B also needs twice the GPU time per request.

**A second opinion.** Two observations point to a better judge of the drafts. The self-check alone ranks the 70B's drafts better than the six-signal cascade (AUC 0.69), and so does agreement between the two open models: when the 8B names the same first file as the 70B, the 70B is right far more often. Ranking drafts by the self-check, plus one when the 8B agrees, with no fitted weights, gives AUC 0.71 and scores

Table 6: Code, with the closed model at minimal reasoning effort (same memory, drafts and probes). Gaps against the full-context reference at the same effort (83.0%) and at default effort (86.0%), with paired bootstrap 95% intervals. "Other folds": each fold's $\lambda$ is chosen on the other folds, removing in-sample selection.

| | | vs. reference, same effort | | vs. reference, default effort | |
|---|---|---|---|---|---|
| Option | Acc. (%) | Gap [95% CI] | Tokens (%) | Gap [95% CI] | Tokens (%) |
| Per-route gate, $\lambda$ chosen in-sample | 80.7 | −2.3 [−5.3, +0.7] | 9.8 | −5.3 [−8.7, −2.3] | 6.8 |
| Per-route gate, $\lambda$ chosen on other folds | 81.0 | −2.0 [−5.0, +0.7] | 9.9 | −5.0 [−8.3, −2.0] | 6.8 |
| Cascade gate | 79.7 | −3.3 [−6.7, −0.3] | 10.0 | −6.3 [−9.7, −3.0] | 6.9 |
| Always escalate, summaries | 83.3 | +0.3 [−2.0, +2.7] | 11.6 | −2.7 [−5.3, +0.0] | 8.0 |

Table 7: Each open model drafting the same questions, with the same team memory and closed-model answers (all but Llama 3.1 8B added after the run). Alone: the open model with team memory. 10%: the cascade at a 10% budget (its share of Muse's tokens). Perfect: escalating exactly the wrong drafts (share). AUC: the cascade's signal. Fast mode: Muse and GPU dollars per thousand questions, and the median wait. Llama 3.3 70B: FP8 on two H100s, code only. Muse Glimmer 30B: bf16 on one H100, reasoning strength low.

| Open model | Alone (%) | 10% budget (%) | Perfect check (%) | AUC | Fast mode |
|---|---|---|---|---|---|
| *Code* | | | | | |
| Llama 3.1 8B | 55.7 | 74.0 (10.0) | 88.3 (12.1) | 0.65 | $11.87, 2.3 s |
| Llama 3.3 70B | 67.0 | 77.7 (9.7) | 90.0 (9.9) | 0.58 | $13.23, 2.6 s |
| Muse Glimmer 30B | 76.0 | 80.3 (6.8) | 89.0 (7.5) | 0.64 | $21.39, 13.2 s |
| *Conversations* | | | | | |
| Llama 3.1 8B | 38.3 | 54.7 (9.4) | 65.7 (10.0) | 0.74 | $5.61, 3.4 s |
| Muse Glimmer 30B | 38.7 | 57.0 (9.9) | 63.3 (9.8) | 0.72 | $13.81, 10.2 s |

81.0% at 10% of the tokens (80.3% with the threshold chosen on the other folds). That is +3.3 points over the cascade (interval −0.3 to +7.0; better in 95% of resamples), and 86.0% at 20%. For the 8B's drafts, with the 70B as the second opinion, the gain is smaller (+1.0, interval −2.7 to +4.7). We designed this rule after looking at these drafts, and it needs both open models running, so we treat it as the direction for the next gate rather than as a result.

Two other probes did not help. Running the self-check on the exact draft that was scored, rather than a fresh one, left its AUC unchanged (0.69 against 0.69), so re-drafting noise was not what limited it. A check that shows the model the issue and the memory's summaries of the file the draft names, and asks whether that file must change, was weaker (AUC 0.60, and 0.53 for the 8B), and adding it to the self-check did not raise accuracy at 10% (78.0%). Fitting weights to a single signal does not help either. Pooled out-of-fold scores carry each fold's intercept, and since folds are whole repositories with different shares of right drafts, the fold rather than the signal decides much of the ranking (`eval/simple_gate.py`).

**Muse Glimmer 30B.** We then ran Meta's open-weight reasoning model, Muse Glimmer 30B (bf16, one H100), the same way on both benchmarks at its lowest reasoning strength. Two things change for the gate. vLLM returns probabilities for the reasoning as well as the answer, so the gate reads only the answer's tokens, and after reasoning those are nearly certain, so the probability features carry little. The self-check and the resamples both reason first. On code, Glimmer's drafts are right on 76.0% of issues, +20.3 points over the 8B (interval +14.7 to +26.0). The cascade at 10% scores 80.3% at 6.8% of the tokens and keeps 77% of questions on the open model (+6.3 over the 8B, interval +1.7 to +11.0), still −5.7 points from the reference. A perfect judge of its drafts would pass (89.0% at 7.5%). On conversations it adds nothing by itself (38.7% against 38.3%, gain +0.3, interval −3.7 to +4.7), as the retrieval analysis predicts.

Reasoning costs time. A Glimmer draft takes 6.1 s at the median against 1.0 s for the 8B. On our cost model, which charges each request a whole H100, fast mode with Glimmer costs $21.39 per thousand code questions, against $11.87 with the 8B and $16.11 for sending every question to the closed model with memory. Glimmer's re-drafts reproduced the scored answer on 81% of code items. It is the best drafter we measured, and at this price it makes fast mode more accurate rather than cheaper.

**Choosing the open model.** On code these runs suggest a simple rule. If every question goes to the closed model with memory, the open model only builds the memory. That is the most accurate setting we measured (86.3% at $16.11 per thousand questions, 9.8 s at the median), and the 8B is enough for it. For the cheapest and fastest answers, the 8B's cascade is the best of the three ($11.87, 2.3 s). Glimmer's cascade loses on all three counts to sending every question to the closed model with memory (80.3%, $21.39 and 13.2 s, against 86.3%, $16.11 and 9.8 s), because each of its drafts

and probes pays for reasoning on the GPU. Where nothing may leave the team's hardware, the open model answers alone, and there Glimmer is the clear choice (76.0% against 67.0% and 55.7%); the release offers this as local mode. The 70B sits in between, and its cascade's gains over the 8B's are not yet distinguishable from chance. On conversations no open model helps much, since retrieval limits every route. In all three runs the memory was built by the 8B.

# 7 Discussion

**Where the cost goes.** We started from the premise that context dominates the bill. For a reasoning model that holds only until the context is compressed. After that, most of each escalation's cost is the closed model's own reasoning, so a 10% budget allows only a few escalations and the gate has to be both accurate and selective. That leaves two levers: send fewer questions, with a better gate, or make each escalation cheaper, for instance with a lower reasoning effort, which moved code close to the bar in our post-hoc run.

**What a gate can know.** The open model's confidence identifies its own wrong answers reasonably well. The oracle's extra advantage comes from knowing when the closed model will also fail, and no signal computed on the draft provides that. Routing studies that report only the cheap model's accuracy, or count calls instead of tokens, may understate this limit.

**The path to the goal.** The 10% goal remains Itara's target, and on code the measurements suggest it is reachable with more than nine days and one GPU. The pass region is not empty: the oracle router reaches 90.3% at 8.8% of the tokens. The open model is what limits the gate. The 8B's draft is right on only 56% of issues, so even a perfect draft check needs 12.1% of the tokens. With the 70B or Glimmer drafting, a perfect check passes, and the cascade at 10% rises from 74.0% to 77.7% and 80.3%. The gate's signal did not improve with the larger models, though, so the main lever now is a better judge of the drafts, such as the second opinion above, rather than a bigger model. The gates were also trained on only about 480 items per fold, and a deployed gateway collects more as a matter of course. On conversations a larger model is not enough, because retrieval caps every route, and better retrieval has to come first. These are expectations to test, not results.

**Why open weights.** The shared cache needs control of the model's attention state, which closed APIs do not expose, and a cache is only reusable by the model that produced it. The layer that handles a team's repeated reading therefore has to be an open-weight model. We built and measured Itara with Llama and Muse Glimmer, but other open families would work as well.

# 8 Limitations

- **Simulated teams.** Teams are repositories and conversations, and the replay simulates engineers. We did not observe real teams.
- **Models.** We used one closed model and three open models, two of them only after the run (Llama 3.3 70B on code, and Muse Glimmer 30B). Results may differ for others.
- **An LLM judge** scores LoCoMo. The hand check agreed on 48 of 50 verdicts, and both disagreements were lenient calls by the judge.
- **A lenient task.** File localisation on well-known repositories rewards prior knowledge; the reference is often right without seeing the file. It is also not patch generation, and we make no claim about repairing code.
- **Pooled memory.** Each repository's memory holds the retrieved files of all its sampled issues, across versions, so a question can retrieve files the reference's prompt did not contain. We did not fix a snapshot per issue.
- **Gate features and selection.** Confidence probes re-draft the answer, and re-drafting under batching does not always reproduce the scored draft. Thresholds come from out-of-fold sweeps on the same items, with no untouched test set. Pooling five folds' predictions under one threshold can blur the ranking, since folds are whole repositories with different intercepts; a deployed gate is a single model and does not have this problem. The live gate does not enforce a token ratio, and it was trained only on file paths and short conversational answers.
- **Post-hoc choices** (budgets, reasoning effort, extra baselines, other open models) were made after seeing results. Choosing an operating point from a sweep on the same items is a mild selection, and our nested check suggests it is small here.
- **Sample size.** With 300 items per benchmark, intervals on gaps are about ±3 points. Item-level bootstrap intervals ignore clustering by repository and conversation.
- **The cost model** is ours: list prices, and an H100 charged per request as if unshared. It counts no idle GPU time, startup, CPU or database. The offline cascade's confidence calls are estimated, not timed (the live run measures them). We compare against simpler retrieval and routing but not against an off-the-shelf prompt compressor [9].
- **Cache scope.** We tested scope-keyed prefix caching on one server. Sharing the cache across servers (LMCache) was not evaluated.

# 9 Conclusion

Itara gives a team one open-weight model, one compressed memory and one permission-scoped KV cache, and calls a closed model only when the open model is unsure. On code, team memory lets the closed model match its full-context accuracy with under a tenth of the context, about a quarter of the tokens and roughly half the cost. The open model's confidence is a usable escalation signal, sharing memory makes

each extra engineer cheaper, and the scoped cache keeps teams apart. On conversations, retrieval rather than routing limits accuracy. The goal we set in advance, matching the closed model at 10% of its tokens, is not reached yet, but the measurements point the way. The oracle passes on code, the open model's accuracy is the main limit, and stronger open models lift the cascade at 10% from 74.0% to 77.7% (Llama 3.3 70B) and 80.3% (Muse Glimmer 30B). Next we plan a better judge of the drafts, gates trained on more questions, better retrieval and cheaper escalations. Every escalation also pairs the open model's draft with the closed model's answer on the team's own questions, which makes the gateway a natural source of data for improving the team's open model.

## Reproducibility

Code, per-item answers for every real-model run, gate replays and analysis notebooks are in the repository (`results/`). Every table and figure in this paper is generated from those files by `paper/generate.py`, which calls no model. Simulated runs used to test the pipeline are excluded from all reported numbers. After the main runs the gateway gained a few small changes (memory for tool requests, ingest that never compresses a page twice, an output limit per call); none of them affects a reported number.

## Disclosure and use of AI assistance

This work was carried out during the Meta Global AI Developer Hackathon (26 October–4 November 2026). The only earlier code in use, about 20 lines of a compression prompt from the author's earlier project, was imported in one marked commit; everything else was written during the event (see `DISCLOSURE.md`). AI coding assistants were used for research, planning, code and writing, and the author reviewed all design decisions, experiments and results. The benchmark data are public (SWE-bench Lite, LoCoMo), and no private or company data was used.

## References


[1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[3] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[5] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[6] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. Auditing prompt caching in language model APIs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 20477–20496, 2025.

[7] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Language model cascades: Token-level uncertainty and beyond. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL `https://research.trychroma.com/context-rot`.

[9] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[12] Andrew Kane. pgvector: Open-source vector similarity search for Postgres. `https://github.com/pgvector/pgvector`, 2021.

[13] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

[17] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025. MLSys 2026.

[18] Llama Team, AI @ Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[19] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[20] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[21] Meta. Meta Model API (Muse Spark). Accessed through the Meta Global AI Developer Hackathon, 2026.

[22] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

[23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[24] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. The early bird catches the leak: Unveiling timing side channels in LLM serving systems. *arXiv preprint arXiv:2409.20002*, 2024.

[25] The PostgreSQL Global Development Group. PostgreSQL documentation: Row security policies. `https://www.postgresql.org/docs/current/ddl-rowsecurity.html`, 2024.

[26] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[28] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[29] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pages 94–109, 2025.

[30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.